SOFTWARE REQUIREMENTS SPECIFICATION

TravelPort

Goibibo-Inspired Full-Stack Travel Booking Portal

DOCUMENT VERSION	DATE	STATUS	CLASSIFICATION
v1.0.0	May 2026	Final	Internal

Table of Contents

Software Requirements Specification — TravelPort v1.0.0

1 Introduction

Purpose, scope, definitions and references

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements for **TravelPort**, a Goibibo-inspired full-stack travel booking portal. It is intended for developers, QA engineers, and project stakeholders as the authoritative specification for system behaviour.

1.2 Scope

TravelPort provides end-to-end travel booking capabilities including: a multi-mode homepage (Flights / Hotels / Buses / Trains / Cabs) with per-mode hero gradients and form state; city autocomplete with 65 Indian cities; a themed DatePicker (no browser format, click-anywhere to open); flight search with Goibibo-style fare popup, date strip, and fare calendar; hotel discovery and reservation; full Bus/Train/Cab search with booking persistence (`TransportSnapshot` JSON column) and extended filter sidebars matching flight depth; digital wallet; coupon system; Saved Traveller quick-fill across all booking pages; toast notifications; resilient error recovery; PDF invoices; and a role-based admin and hotel-manager portal.

900+
Seed Flights

60+
Hotels (12 cities)

225+
Features Delivered

1.3 Definitions & Acronyms

TERM	DEFINITION
`JWT`	JSON Web Token — stateless bearer token for API authentication (15-minute TTL)
`SRS`	Software Requirements Specification — this document
`EF Core`	Entity Framework Core — ORM used for .NET database access
`DTO`	Data Transfer Object — contract between API layers
`SMTP`	Simple Mail Transfer Protocol — used for transactional email via Office365
`BCrypt`	Password hashing algorithm (cost factor 12) used for credential storage
`CI/CD`	Continuous Integration / Continuous Deployment via GitHub Actions

1.4 References

- **ARCHITECTURE.md** — System architecture and layer design
- **API_DOCUMENTATION.md** — Full REST API reference
- **DATABASE_DESIGN.md** — Entity-relationship diagram and schema
- **SECURITY_GUIDE.md** — Authentication, authorization and security policies

2 System Overview

Product perspective, user classes and operating environment

2.1 Product Perspective

TravelPort is a standalone web application. It uses a React 18 SPA frontend communicating with a .NET 8 REST API backed by SQL Server. All data flows through the API gateway; no direct DB access from the client.

SYSTEM LAYERS

Client Layer — React 18 + TypeScript + Tailwind + Redux Toolkit (Vite 6)

↕ HTTPS / REST

API Gateway — .NET 8 Web API | JWT Auth | Rate Limiting (Swagger/OpenAPI)

↕

Application Services — FlightService | HotelService | BookingService | WalletService | AdminService

↕

Persistence — EF Core 8 | SQL Server Express | ICacheService (in-memory / Redis)

↕

Infrastructure — SMTP Email | Razorpay | Duffel (optional) | Nginx Reverse Proxy

2.2 User Classes

ROLE	DESCRIPTION	ACCESS LEVEL
Guest	Unauthenticated visitor; can search flights, hotels, buses, trains, cabs and validate coupons	Read-only search
User	Registered traveller; can book all transport modes, pay via wallet/card/UPI, cancel, download invoices, use SavedTravellerPicker, and manage wallet & saved cards	Full self-service
Admin	System operator; manages analytics, users (block/unblock), all bookings	Full system

ROLE	DESCRIPTION	ACCESS LEVEL
Hotel Manager	Hotel-scoped operator; manages own hotel's bookings, room types, amenities, images and profile via `/hotel/*` portal. JWT includes `hotelId` claim — all queries automatically scoped.	Hotel-scoped portal

2.3 Operating Environment

DEVELOPMENT

Windows / macOS / Linux

Node 20+ · .NET 8 SDK · SQL Server Express

PRODUCTION (DOCKER)

Any Docker host

docker-compose · SQL Server 2022 container · Nginx SPA+proxy

BROWSERS

Chrome 90+ · Firefox 88+ · Safari 14+

Mobile Chrome/Safari supported

CI/CD

GitHub Actions

pre-commit test gate · Docker Hub registry · approval-gated image publish

3 Functional Requirements

Detailed behaviour for each module

3.1 Authentication Module

ID	REQUIREMENT	PRIORITY
FR-AUTH-01	System shall allow new users to register with email, password and full name	High
FR-AUTH-02	System shall issue a JWT access token (15 min TTL) and refresh token (7 day TTL) on login	High
FR-AUTH-03	System shall auto-rotate refresh tokens; old token revoked after use	High
FR-AUTH-04	System shall support forgot-password flow: time-limited reset link via email (1 hour TTL)	High
FR-AUTH-05	Passwords must be hashed with BCrypt cost factor 12 before storage	High
FR-AUTH-06	Admin role must be enforced via JWT claims on all /admin/* routes	High

3.2 Flight Booking Module

ID	REQUIREMENT	PRIORITY
FR-FLT-01	System shall search flights by origin, destination, date, passengers and class; cache 5 min	High
FR-FLT-02	Search results shall be filterable by airline, stops, departure time, and price range	High
FR-FLT-03	System shall display fare families (Economy/Premium Economy/Business) with distinct pricing	High
FR-FLT-04	System shall collect per-traveller name, age, gender and ID type before booking	High
FR-FLT-	Booking shall atomically deduct wallet (if used) and decrement available seats	High

ID	REQUIREMENT	PRIORITY
FR-FLT-06	System shall apply validated coupons and reduce FinalAmount accordingly	Medium
FR-FLT-07	System shall send booking-confirmed email with PDF e-ticket attached	High
FR-FLT-08	System shall allow cancellation with wallet refund for eligible bookings	High
FR-FLT-09	BookingExpiryWorker shall auto-cancel Pending bookings older than 30 minutes	Medium

3.3 Hotel Booking Module

ID	REQUIREMENT	PRIORITY
FR-HTL-01	System shall search hotels by city, check-in/out, guests, star rating and sort order; cache 5 min	High
FR-HTL-02	Hotel results shall support filter by star rating, price range, and amenities	High
FR-HTL-03	Booking shall capture guest name, email and phone number	High
FR-HTL-04	System shall generate hotel invoice PDF with stay dates, room details, GST breakdown	High
FR-HTL-05	Booking references shall use HT2026XXXXXX prefix format	Low

3.4 Transport Module (Bus / Train / Cab)

Bus, Train and Cab search uses deterministic in-memory providers — consistent results per route. Full end-to-end booking is implemented via `BusService`, `TrainService`, and `CabService` with coupon, wallet, email, and PDF ticket support. Journey details are stored in a nullable `TransportSnapshot` JSON column on the `Booking` entity.

> ✓ **Full Booking Flow Implemented**
>
> Bus (14 operators, 10–22 results/route) · Train (39 named trains, 5 classes, 6–15 results/route) · Cab (Ola/Uber/Meru/Zoom, distance-based pricing). Each has a dedicated booking page, backend service, PDF ticket generator, HTML email template, and `BookingDetailPage` transport view. Booking references use BU/TR/CB prefixes.

> ⚙ **Extended Filter Sidebars (Phase 12)**

All three listing pages now have filter sidebars on par with Flights and Hotels: sort tabs (Cheapest/Fastest/Earliest or Top Rated), departure time slot buttons (🌙 🌅 ☀️ 🌆), multi-select type/class/provider checkboxes, min driver rating (Cabs), max price, and a Clear all control.

3.5 Payment & Wallet

ID	REQUIREMENT	PRIORITY
`FR-PAY-01`	System shall integrate Razorpay checkout; falls back to mock payment in dev mode	High
`FR-PAY-02`	Each user shall have exactly one wallet, created on registration	High
`FR-PAY-03`	Wallet top-up maximum is ₹1,00,000 per transaction	High
`FR-PAY-04`	Wallet deduction and booking creation shall be committed atomically (single UoW)	High
`FR-PAY-05`	WalletTransaction shall record every credit/debit with description and reference ID	High
`FR-PAY-06`	System shall support 11 coupons (flight-specific and hotel-specific variants)	Medium
`FR-PAY-07`	User may save credit/debit cards (last 4 digits only); one card may be set as default	High
`FR-PAY-08`	At booking time, user selects Wallet, Saved Card, or New Card as payment method	High
`FR-PAY-09`	Payment page shall support Card (live preview), UPI (QR + timer), Net Banking (bank picker + timer)	High
`FR-PAY-10`	Cancellation shall atomically credit 90% of final amount to wallet and persist immediately	High
`FR-PAY-11`	Booking confirmation email shall be sent to the contact email entered at booking, not the account email	Medium

3.6 Admin Panel

- **Dashboard:** Monthly revenue chart, bookings by type, bookings by status
- **Users:** Paginated user list with block/unblock action
- **Bookings:** All bookings with status filter; manual cancellation
- **Coupons:** Full CRUD — create, edit, toggle active, delete coupons

- **Analytics:** `GET /admin/analytics` — real SQL aggregations (not mocked)

4 Non-Functional Requirements

Performance, security, reliability and usability

Performance

Flight/hotel search responds in < 500 ms (cache hit < 50 ms). API supports 100 concurrent users on a single 2-core server.

Security

JWT (15 min) + Refresh Tokens (7 days). BCrypt cost 12. Secrets in env vars only. HTTPS enforced in production. SQL injection prevention via EF Core parameterization.

Reliability

Background workers (BookingExpiryWorker, RefreshTokenCleanupWorker) recover gracefully on shutdown. DataSeeder is idempotent — existing data preserved on restart. Commits and CI are gated by automated backend and frontend tests.

Usability

Goibibo-style UI with skeleton loaders, toast notifications, route-level error recovery, accessible form controls, and keyboard navigation.

5 Key Use Cases

Primary user journeys through the system

UC-01: Book a Flight

1. **Search**
   User enters origin, destination, date and passenger count on the home page
2. **Browse**
   Results load with airline filters, stop filters, sort tabs (Cheapest/Fastest/Earliest)
3. **Select Fare**
   User clicks VIEW FARES; fare-family popup shows Economy / Premium Economy / Business

4. **Traveller Details**

   User fills per-traveller name, age, gender and ID number

5. **Apply Coupon**

   Optional coupon code reduces FinalAmount; discount displayed in summary

6. **Pay**

   Razorpay checkout (or wallet deduction); booking confirmed atomically

7. **Confirmation**

   Email with PDF e-ticket sent; booking appears in My Bookings with FL prefix